

ESP32-S3 & LSM6DSOX: Guide för Datadump via ESP-IDF

Detta dokument innehåller en komplett, ren och registerbaserad implementering i **ESP-IDF (v5.x)** för att initiera I2C-bussen på en **ESP32-S3 N16R8** och kontinuerligt dumpa rådata samt grundläggande skalad data från accelerometer och gyroskop i din **ST LSM6DSOX**. Koden är skriven i ren C och anpassad för PlatformIO i VS Code.

1. Konfiguration av platformio.ini

```
[env:esp32-s3-devkitc-1]
platform = espressif32
framework = espidf
board = esp32-s3-devkitc-1
monitor_speed = 115200

board_upload.flash_size = 16MB
board_build.partitions = default_16MB.csv
build_flags =
    -DBOARD_HAS_PSRAM
    -mfix-esp32-s3-psram-cache-issue
    -DCONFIG_SPIRAM_MODE_OCT=1
```

2. Hårdvarukoppling (I2C)

LSM6DSOX Pin	ESP32-S3 GPIO	Beskrivning
VIN / VCC	3V3	Strömförsörjning
GND	GND	Gemensam jord
SDA	GPIO 11	I2C Data-lina
SCL	GPIO 12	I2C Klock-lina

3. Källkod (src/main.c)

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/i2c.h"
#include "esp_log.h"

static const char *TAG = "LSM6DSOX_DUMP";

#define I2C_MASTER_NUM          I2C_NUM_0
#define I2C_MASTER_SDA_IO      11
#define I2C_MASTER_SCL_IO      12
#define I2C_MASTER_FREQ_HZ     400000
#define I2C_MASTER_TX_BUF_DISABLE 0
#define I2C_MASTER_RX_BUF_DISABLE 0

// Ändra till 0x6B om kretskortets folie/bygling är satt till den högre adressen
#define LSM6DSOX_I2C_ADDR      0x6A

// Registeradresser
#define LSM6DSOX_REG_WHO_AM_I    0x0F
#define LSM6DSOX_REG_CTRL1_XL    0x10
#define LSM6DSOX_REG_CTRL2_G     0x11
#define LSM6DSOX_REG_OUTX_L_G    0x22
#define LSM6DSOX_REG_OUTX_L_A    0x28

esp_err_t lsm6dsox_register_read(uint8_t reg_addr, uint8_t *data, size_t len) {
    return i2c_master_write_read_device(I2C_MASTER_NUM, LSM6DSOX_I2C_ADDR,
                                         &reg_addr, 1, data, len,
                                         pdMS_TO_TICKS(1000));
}

esp_err_t lsm6dsox_register_write(uint8_t reg_addr, uint8_t data) {
    uint8_t write_buf[2] = {reg_addr, data};
    return i2c_master_write_to_device(I2C_MASTER_NUM, LSM6DSOX_I2C_ADDR,
                                       write_buf, 2, pdMS_TO_TICKS(1000));
}

void i2c_master_init(void) {
    i2c_config_t conf = {
        .mode = I2C_MODE_MASTER,
        .sda_io_num = I2C_MASTER_SDA_IO,
        .scl_io_num = I2C_MASTER_SCL_IO,
        .sda_pullup_en = GPIO_PULLUP_ENABLE,
        .scl_pullup_en = GPIO_PULLUP_ENABLE,
        .master.clk_speed = I2C_MASTER_FREQ_HZ,
    };
    ESP_ERROR_CHECK(i2c_param_config(I2C_MASTER_NUM, &conf));
    ESP_ERROR_CHECK(i2c_driver_install(I2C_MASTER_NUM, conf.mode,
                                       I2C_MASTER_RX_BUF_DISABLE, I2C_MASTER_TX_BUF_DISABLE, 0));
    ESP_LOGI(TAG, "I2C master initierad framgångsrikt.");
}

void app_main(void) {
    i2c_master_init();
    vTaskDelay(pdMS_TO_TICKS(100));

    uint8_t who_am_i = 0;
    esp_err_t ret = lsm6dsox_register_read(LSM6DSOX_REG_WHO_AM_I, &who_am_i, 1);

    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "Kunde inte kommunicera med sensorn via I2C.");
        return;
    }

    if (who_am_i != 0x6C) {
        ESP_LOGE(TAG, "Felaktigt WHO_AM_I id: 0x%02X (Förväntade 0x6C).",
        who_am_i);
    }
}
```

```

        return;
    }
    ESP_LOGI(TAG, "LSM6DSOX identifierad! (WHO_AM_I: 0x%02X)", who_am_i);

    // Initiera Accelerometer: 104 Hz, +/- 2g känslighet (CTRL1_XL = 0x40)
    ESP_ERROR_CHECK(lsm6dsox_register_write(LSM6DSOX_REG_CTRL1_XL, 0x40));
    // Initiera Gyroskop: 104 Hz, +/- 250 dps känslighet (CTRL2_G = 0x40)
    ESP_ERROR_CHECK(lsm6dsox_register_write(LSM6DSOX_REG_CTRL2_G, 0x40));

    ESP_LOGI(TAG, "Sensorer aktiverade. Startar datadump...");

    int16_t raw_ax, raw_ay, raw_az;
    int16_t raw_gx, raw_gy, raw_gz;
    uint8_t data_buf[6];

    while (1) {
        if (lsm6dsox_register_read(LSM6DSOX_REG_OUTX_L_A, data_buf, 6) ==
ESP_OK) {
            raw_ax = (int16_t)((data_buf[1] << 8) | data_buf[0]);
            raw_ay = (int16_t)((data_buf[3] << 8) | data_buf[2]);
            raw_az = (int16_t)((data_buf[5] << 8) | data_buf[4]);
        }

        if (lsm6dsox_register_read(LSM6DSOX_REG_OUTX_L_G, data_buf, 6) ==
ESP_OK) {
            raw_gx = (int16_t)((data_buf[1] << 8) | data_buf[0]);
            raw_gy = (int16_t)((data_buf[3] << 8) | data_buf[2]);
            raw_gz = (int16_t)((data_buf[5] << 8) | data_buf[4]);
        }

        float ax = (float)raw_ax / 16384.0f;
        float ay = (float)raw_ay / 16384.0f;
        float az = (float)raw_az / 16384.0f;

        float gx = (float)raw_gx / 114.0f;
        float gy = (float)raw_gy / 114.0f;
        float gz = (float)raw_gz / 114.0f;

        ESP_LOGI(TAG, "ACCEL [g] | X: %5.2f Y: %5.2f Z: %5.2f || GYRO [dps] | X:
%5.1f Y: %5.1f Z: %5.1f",
                ax, ay, az, gx, gy, gz);

        vTaskDelay(pdMS_TO_TICKS(200));
    }
}

```